ETH
Swiss Federal Institute of Technology Zurich

Seminar for Statistics

Department of Mathematics

Semester Paper (No. 2) Spring Semester 2026

Basil Frei

Order Matters, Randomness Too: Data Sampling Strategies for Finite-Sum Minimization

Submission Date: July 20, 2026

Co-Advisor Dr. Peter Whalley
Advisor: Prof. Dr. Yuansi Chen

Abstract

We compare the four stochastic first-order methods stochastic gradient descent (SGD), random reshuffling (RR), shuffle-once (SO) and incremental gradient (IG) for finite-sum minimization. They differ only in *how* they traverse the data. We first recreate strongly convex ℓ^2-regularized logistic regression experiments from prior work on real-sim, w8a and RCV1. We then test whether these findings translate to the non-convex case. To this end we train neural networks on Bike Sharing, Concrete Compressive Strength and a subsample of MNIST, for each an affine network and a two-hidden-layer ReLU network. Under strong convexity our results match the reference. RR and SO converge at the same speed and clearly beat SGD, while IG is inconsistent and usually trails the shuffling methods. We also reproduce the predicted epoch-wise oscillations of the without-replacement schemes. In the non-convex setting the ranking changes. SGD catches up to RR, SO falls behind and IG performs worst. We attribute this to fresh stochasticity, which RR and SGD keep injecting while SO and IG repeat one fixed trajectory. Order matters and so does randomness. RR was tied for best in every experiment and never worse than any competitor. This makes RR a safe first choice for finite-sum minimization.

1 Introduction

In this work we contrast four different stochastic first-order gradient descent variants to solve *finite-sum minimization* problems. In such tasks the objective function takes the form of an average of n per-sample losses with the goal of finding parameters minimizing this average over an (often very large) dataset. Finite-sum minimization problems are of central importance in machine learning, since empirical risk minimization naturally leads to objectives of this form. Inspired by the work of Mishchenko et al. (2020) we re-implement a selection of their experiments and additionally test whether their results translate from a *strongly convex* setup to a generally *non-convex* case by extending the empirical analysis to simple neural networks.

Since the datasets considered in practice often tend to be very large, finite-sum minimization problems are mostly tackled using *stochastic* variants of the well-known gradient descent algorithm (GD). This is due to the fact that taking the gradient of a sum of n sample-wise losses requires computing the gradient of each of the n individual losses. The computational cost of each GD iteration therefore grows linearly with the dataset size. When n is large, repeated full passes over the dataset become computationally prohibitive and may exceed available memory. Furthermore, Bengio (2012) notes that computing the full gradient at every iteration limits the algorithm's ability to explore the optimization landscape. He argues that this is the case as the gradient is only the direction of steepest descent locally and therefore does not necessarily lead along a path toward a global minimizer, particularly in the ubiquitous non-convex setting.

A common way to overcome these issues is to randomly sample data points and compute only the gradients of the corresponding per-sample losses. This gives rise to so-called stochastic first-order methods. According to Bengio (2012), stochastic variants of GD scale much more favorably to large datasets, with their effective convergence speed increasing almost linearly beyond a certain dataset size. Mini-batch variants further improve efficiency by enabling parallel computation on modern hardware.

The variants we consider here differ in *how* they *order* and *sample* data: with vs. without replacement, deterministically vs. probabilistically and with what cadence.[1] In fact, as Mishchenko et al. (2020) show in their Theorem 1, the specific replacement and traversal strategy used can provably make a difference on the convergence speed an algorithm achieves.

1.1 Related Work

To the best of our knowledge, it was first brought to public attention during the 2009 ASA SLDS conference[2] that a method today known as random reshuffling (RR) seemed to converge faster than simple stochastic gradient descent (SGD) does (Bottou 2009). Since then RR has become standard practice in the training of deep neural networks (Sun 2020). PyTorch's `DataLoader` class, for instance, exposes a `shuffle` argument

[1] We use *order* and *ordering* loosely to also cover with-replacement sequences of the dataset, where a data point may repeat or be omitted within an epoch.

[2] A yearly conference organized by the Statistical Learning Data Science (SLDS) section of the American Statistical Association (ASA)

that draws a fresh permutation of the data at the start of every epoch when set to `True` (PyTorch Contributors 2026b), making RR a one-flag default.

While theoretical convergence guarantees for SGD have been well-known for a long time, the analysis of shuffling methods like RR and shuffle-once (SO) remained a harder nut to crack. HaoChen and Sra (2018) were among the first to show that RR provably beats SGD once enough epochs have passed. Gürbüzbalaban et al. (2021) gave a complementary account of why reshuffling helps under a related set of assumptions. Using novel techniques, Mishchenko et al. (2020), the main reference for this paper, were able to prove results for RR and SO that have a much better dependence on problem constants like the dataset size n and the condition number κ.[3] Additionally, they were the first to prove results for SO that did not ignore its inherent randomness. Previously, results for SO were merely applications of the worst-case bounds known for incremental gradient (IG), the deterministic sibling of SO. The asymptotic convergence of IG was already established at the start of the century (Bertsekas and Tsitsiklis 2000).

Malinovsky et al. (2023) combined RR with variance reduction, giving the first analysis of SVRG under random reshuffling. More recently, Liu (2026) showed RR dominates SGD under any reasonable stepsize after any finite number of epochs in smooth convex optimization. Earlier theory only guaranteed RR beats SGD once the stepsize is small enough or the epoch count high enough.

2 Mathematical Setup

In this section we lay out the mathematical framework underlying our experiments: the finite-sum minimization problem and its two instantiations (ℓ^2-regularized logistic regression and neural networks), the four stochastic first-order methods we compare and their extension to mini-batch variants with varying stepsize.

2.1 Problem Formulation

The optimization problems tackled here all fall under the following regime: we are given a dataset of n points $(\boldsymbol{a}_i, b_i)_{i=1}^n$ where the features $\boldsymbol{a}_i \in \mathbb{R}^d$ and the labels b_i belong to a task-dependent label space $\mathcal{B} \subseteq \mathbb{R}$, for instance $\{-1, +1\}$ or $\{0, ..., 9\}$. The goal is to find a predictor $\mathbb{R}^d \to \mathbb{R}^{d_{\text{out}}}$ parametrized by $\boldsymbol{x} \in \mathbb{R}^p$ whose quality on a sample $(\boldsymbol{a}_i, b_i)$ is measured by a per-sample loss $f_i(\boldsymbol{x})$. For regression $d_{\text{out}} = 1$ and in classification we have $d_{\text{out}} = |\mathcal{B}|$,[4] where the predictor outputs one score per class. Seeking to minimize the average loss incurred across the whole dataset yields a finite-sum minimization problem

$$\min_{\boldsymbol{x} \in \mathbb{R}^p} f(\boldsymbol{x}) := \frac{1}{n} \sum_{i=1}^{n} f_i(\boldsymbol{x}). \tag{1}$$

We instantiate (1) in two settings: strongly convex ℓ^2-regularized logistic regression as well as generally non-convex neural network regression and classification.

ℓ^2-Regularized Logistic Regression. Here

$$f_i(\boldsymbol{x}) = \log\left(1 + \exp(-b_i \boldsymbol{a}_i^\top \boldsymbol{x})\right) + \frac{\lambda}{2} \|\boldsymbol{x}\|_2^2, \tag{2}$$

where $b_i \in \{-1, +1\}$ and $\lambda > 0$ is the regularization strength. Depending on the specific dataset, the labels b_i may instead be given in binary $\{0, 1\}$ form (or some other encoding), in which case we first apply a change of variables to map them onto $\{-1, +1\}$ before proceeding. Since the unregularized logistic loss is convex and $\frac{\lambda}{2} \|\boldsymbol{x}\|_2^2$ is λ-strongly convex, the f_i and the finite-sum objective f are λ-strongly convex. Moreover, each f_i is L_i-smooth with $L_i = \|\boldsymbol{a}_i\|_2^2 / 4 + \lambda$, so that f itself is L_f-smooth with $L_f = L + \lambda$ where $L = \|\boldsymbol{A}\|_2^2 / (4n)$ is the spectral smoothness of the unregularized logistic loss and $\boldsymbol{A} = [\boldsymbol{a}_1, ..., \boldsymbol{a}_n]^\top$ the data matrix.

Neural Networks. We consider two network architectures. Our more expressive architecture is a ReLU network with two hidden layers,

$$\mathfrak{nn}_{\boldsymbol{x}} : \boldsymbol{a} \mapsto \boldsymbol{W}^{(3)} \operatorname{ReLU}(\boldsymbol{W}^{(2)} \operatorname{ReLU}(\boldsymbol{W}^{(1)} \boldsymbol{a} + \boldsymbol{c}^{(1)}) + \boldsymbol{c}^{(2)}) + \boldsymbol{c}^{(3)}, \tag{3}$$

[3] The ratio between the smoothness constant and the strong convexity constant

[4] Binary ℓ^2-regularized logistic regression (see Equation (2)) is an exception to this convention. It uses a single score $\boldsymbol{a}_i^\top \boldsymbol{x}$ with $d_{\text{out}} = 1$, since the second class probability is determined by the first.

where the weight matrices and bias vectors $(\boldsymbol{W}^{(l)}, \boldsymbol{c}^{(l)})_{l=1}^{3}$ are flattened into $\boldsymbol{x} \in \mathbb{R}^p$ and ReLU acts component-wise as the activation function $z \mapsto \max(0, z)$. As a baseline we also consider a network with no hidden layers, i.e., a single affine map

$$\mathfrak{nn}_{\boldsymbol{x}} : \boldsymbol{a} \mapsto \boldsymbol{W}^{(1)}\boldsymbol{a} + \boldsymbol{c}^{(1)}. \tag{4}$$

Again we flatten the parameters $(\boldsymbol{W}^{(1)}, \boldsymbol{c}^{(1)})$ into $\boldsymbol{x} \in \mathbb{R}^p$. For the *regression* tasks we choose to measure the prediction quality by the ℓ^2-regularized mean squared error

$$f_i(\boldsymbol{x}) = \left(\mathfrak{nn}_{\boldsymbol{x}}(\boldsymbol{a}_i) - b_i\right)^2 + \frac{\lambda}{2}\left\|\boldsymbol{x}\right\|_2^2. \tag{5}$$

For $|\mathcal{B}|$-*class classification* $\mathfrak{nn}_{\boldsymbol{x}}(\boldsymbol{a}_i) \in \mathbb{R}^{|\mathcal{B}|}$ outputs a vector of raw logits for which

$$f_i(\boldsymbol{x}) = -\log\left(\frac{\exp\left(\mathfrak{nn}_{\boldsymbol{x}}(\boldsymbol{a}_i)_{b_i}\right)}{\sum_{\beta\in\mathcal{B}}\exp\left(\mathfrak{nn}_{\boldsymbol{x}}(\boldsymbol{a}_i)_{\beta}\right)}\right) + \frac{\lambda}{2}\left\|\boldsymbol{x}\right\|_2^2, \tag{6}$$

the ℓ^2-regularized softmax cross-entropy loss, is used to assess model quality. With the affine architecture in Equation (4), $\mathfrak{nn}_{\boldsymbol{x}}$ is linear in $\boldsymbol{x}$. Hence, both per-sample losses (defined by Equations (5) and (6)) reduce to λ-strongly convex functions of $\boldsymbol{x}$. Being an average of the f_i, the two finite-sum objectives are λ-strongly convex, too. By the same argument, since $\mathfrak{nn}_{\boldsymbol{x}}$ is linear in $\boldsymbol{x}$ and both sample-wise loss functions are themselves smooth in the network output, the two average losses are also smooth in the affine setting.

With the two-hidden-layer architecture of Equation (3), the composition with ReLU is no longer linear in $\boldsymbol{x}$ and the f_i are not convex in general. Consequently, also the objectives are generally non-convex.

2.2 Stochastic First-Order Methods

Using the problem setups laid out above, we compare four different stochastic first-order methods on them. For clarity, we first present each scheme in its single-sample form, before generalizing to mini-batches of size $\tau > 1$ in Section 2.3. The algorithms all share the same basic structure: at every epoch t, the parameter vector is updated via n successive gradient steps, one per data point, following some order of the dataset:

$$\boldsymbol{x}_t^{i+1} = \boldsymbol{x}_t^i - \gamma\nabla f_{\pi_i}(\boldsymbol{x}_t^i), \quad i = 0, ..., n-1, \quad \boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^n, \tag{7}$$

where $\gamma > 0$ is the stepsize and $\pi_0, ..., \pi_{n-1}$ is an ordering of $\{1, ..., n\}$ specific to each method. The algorithms considered here differ in *how* this sequence is chosen *within* and *across* epochs. This distinction is the central object of study of our work.

The most straightforward stochastic first-order method samples a new data point independently and uniformly *with* replacement at every step. In other words, at step i of epoch t index $\pi_i \in \{1, ..., n\}$ is chosen with probability $1/n$, independently of all previous draws. This is called **stochastic gradient descent (SGD)** (see Algorithm 1). Crucially, since sampling is with replacement, some per-sample losses may be visited multiple times within a single epoch while others are never considered at all.

A variant called **random reshuffling (RR)** enforces seeing every sample exactly once per epoch by drawing its data points independently and uniformly *without* replacement. This is equivalent to sampling a permutation $\pi_0, ..., \pi_{n-1}$ of $\{1, ..., n\}$ uniformly at random and independently across epochs, at the start of every epoch. Then the gradients of the per-sample losses are taken along this permutation (see Algorithm 2).

The remaining two methods considered here are **shuffle-once (SO)**, which samples its permutation also uniformly, but only *once* and uses it for each epoch (see Algorithm 3) and **incremental gradient (IG)**, where the gradients are computed along a *fixed* order of the dataset (see Algorithm 4), for instance following the order in which the data is stored.

Algorithm 1: Stochastic Gradient Descent (SGD)

Input: Stepsize $\gamma > 0$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T
for *epochs* $t = 0, ..., T-1$ **do**
  for *steps* $i = 0, ..., n-1$ **do**
    Sample a data point $\pi_i \in \{1, ..., n\}$
    $\boldsymbol{x}_t^{i+1} = \boldsymbol{x}_t^i - \gamma \nabla f_{\pi_i}(\boldsymbol{x}_t^i)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^n$
end

Algorithm 2: Random Reshuffling (RR)

Input: Stepsize $\gamma > 0$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T
for *epochs* $t = 0, ..., T-1$ **do**
  Sample a permutation $\pi_0, ..., \pi_{n-1}$ of $\{1, ..., n\}$
  for *steps* $i = 0, ..., n-1$ **do**
    $\boldsymbol{x}_t^{i+1} = \boldsymbol{x}_t^i - \gamma \nabla f_{\pi_i}(\boldsymbol{x}_t^i)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^n$
end

Algorithm 3: Shuffle-Once (SO)

Input: Stepsize $\gamma > 0$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T
Sample a permutation $\pi_0, ..., \pi_{n-1}$ of $\{1, ..., n\}$
for *epochs* $t = 0, ..., T-1$ **do**
  for *steps* $i = 0, ..., n-1$ **do**
    $\boldsymbol{x}_t^{i+1} = \boldsymbol{x}_t^i - \gamma \nabla f_{\pi_i}(\boldsymbol{x}_t^i)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^n$
end

Algorithm 4: Incremental Gradient (IG)

Input: Stepsize $\gamma > 0$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T
for *epochs* $t = 0, ..., T-1$ **do**
  for *steps* $i = 0, ..., n-1$ **do**
    $\boldsymbol{x}_t^{i+1} = \boldsymbol{x}_t^i - \gamma \nabla f_{i+1}(\boldsymbol{x}_t^i)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^n$
end

From Algorithm 1 to Algorithm 4, the point at which stochasticity enters the algorithm moves progressively further outward: from resampling at every single iteration (SGD), to resampling once per epoch (RR), to sampling only once before training begins (SO), to no randomness at all (IG).

2.3 Mini-Batch and Varying Stepsize Variants

In practice and in the experiments of Mishchenko et al. (2020), Algorithms 1, 2, 3 and 4 are employed in a slightly adjusted form, taking gradient steps in *mini-batches* of size $\tau > 1$ rather than single samples. Mini-batches are subsequences of the data, grouped together so that a single gradient step can be computed over all of them at once. This offers two complementary benefits. Statistically, averaging the gradients of τ samples reduces the variance of the resulting gradient estimator by a factor of roughly τ compared to a single-sample estimate (Bottou et al. 2018), yielding a less noisy update direction at each step. Computationally, although this should increase the per-iteration cost at first glance, parallelization on state-of-the-art hardware often decreases the overall wall-clock training time instead (Bengio 2012).

Each algorithm still determines a sweep through the data exactly as in Section 2.2, but additionally partitions it into $m := \lceil n/\tau \rceil$ contiguous mini-batches $S_0, ..., S_{m-1}$. For RR, SO and IG, this ordering is some (randomly drawn or deterministic) permutation $\pi_0, ..., \pi_{n-1}$ of $\{1, ..., n\}$ and the algorithm groups the indices into the sequences $S_j = (\pi_{j\tau}, ..., \pi_{(j+1)\tau-1})$ for $j = 0, ..., m-2$ and $S_{m-1} = (\pi_{(m-1)\tau}, ..., \pi_{n-1})$. We note here that the last mini-batch S_{m-1} may contain fewer than τ samples if τ does not divide n, specifically $|S_{m-1}| = n - (m-1)\tau \leq \tau$.

Since sampling *with* replacement never depletes the pool of indices left to choose, this issue does not arise for SGD. The ordering is instead an i.i.d. sequence $\pi_0, ..., \pi_{m\tau-1}$ drawn uniformly from $\{1, ..., n\}$ and the algorithm groups the indices analogously into $S_j = (\pi_{j\tau}, ..., \pi_{(j+1)\tau-1})$ for $j = 0, ..., m-1$. Every mini-batch, including the last one, always has exactly τ elements.

Writing

$$g_\nu^j(\boldsymbol{x}) := \frac{1}{\nu} \sum_{i \in S_j} \nabla f_i(\boldsymbol{x}) \tag{8}$$

for the mini-batch gradient estimator with normalization constant $\nu > 0$, all four methods have the common update scheme

$$\boldsymbol{x}_t^{j+1} = \boldsymbol{x}_t^j - \gamma g_\nu^j(\boldsymbol{x}_t^j), \quad j = 0, ..., m-1, \quad \boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^m. \tag{9}$$

One subtlety arises in choosing ν. The natural choice is the mini-batch-*dependent* normalization $\nu = |S_j|$, i.e., normalizing each mini-batch gradient by the length of its mini-batch.[5] For SGD, this choice simply gives $\nu = \tau$, since every mini-batch always has exactly τ elements as noted above. For RR, SO and IG, however, the same mini-batch-dependent choice $\nu = |S_j|$ is problematic, since the last batch S_{m-1} may be shorter than the rest. Normalizing each mini-batch gradient by its *own* (potentially shorter) length breaks a structural property that these methods rely on for their convergence guarantees. Shaw and Whalley (2025) mention in their Remark 4.11 that mini-batch-dependent normalization empirically ruins the higher-order convergence of RR. In the case of SO, we observed a similar effect, where mini-batch-dependent normalization led to noticeably worse asymptotic performance. To avoid these issues, Shaw and Whalley (2025) instead recommend a constant, mini-batch-*independent* normalization convention that divides every batch by the full batch size τ. We adopt the same idea here but use $\nu = n/m$ (the *average* batch size across an epoch) for all our *without*-replacement methods, including the possibly shorter last batch S_{m-1}. Since ν and τ differ only by the fixed factor $\tau m/n \approx 1$, this only amounts to a global rescaling of the effective stepsize and does not affect the underlying argument.

A further practical consideration is the choice of stepsize γ itself. With a *constant* stepsize, SGD, for instance, does not converge to the true minimum $\boldsymbol{x}^*$ (if it exists) in a strict topological sense. Since a fresh mini-batch of indices is sampled with replacement at every step, the variance of the SGD gradient estimator never vanishes, even as $\boldsymbol{x}_t^j$ approaches $\boldsymbol{x}^*$. Instead, under suitable assumptions the iterates settle into a convergence neighborhood of $\boldsymbol{x}^*$ whose radius scales with $\sqrt{\gamma}$ (see, e.g., Bottou et al. (2018, Theorem 4.6) or Gower et al. (2019, Theorem 3.1)). Driving this radius to zero requires a stepsize that decreases across iterations, following for example a Robbins–Monro-type schedule with $\sum \gamma = \infty$ and $\sum \gamma^2 < \infty$ (Robbins and Monro 1951). Although RR, SO and IG are less severely affected by this, they still need diminishing stepsizes in order to converge (see, e.g., Ying et al. (2019, Theorem 1 and Corollary 2)).

We therefore let the stepsize optionally vary with t and j, writing γ_t^j in place of the constant γ used above:

$$\boldsymbol{x}_t^{j+1} = \boldsymbol{x}_t^j - \gamma_t^j \, g_\nu^j(\boldsymbol{x}_t^j), \quad j = 0, \ldots, m-1, \quad \boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^m. \tag{10}$$

With this notation in place, Algorithms 5, 6, 7 and 8 give the mini-batch versions of SGD, RR, SO and IG allowing for a non-constant stepsize schedule. In this work, we refer to the algorithms' mini-batch-varying-stepsize formulations even though we use the abbreviations of their single-sample-constant-stepsize siblings, i.e., when talking about RR, for instance, we actually mean Algorithm 6.

3 Experimental Setup

This section details the choices we make to reach our results in Section 4. We discuss the datasets used, how convergence curves are aggregated and reported and what hyperparameter choices we make for both the ℓ^2-regularized logistic regression and neural network experiments. Code and cached results reproducing every figure in this paper, together with additional experiments on further datasets not discussed here, are available at https://github.com/basilfrei/order-matters-randomness-too.

3.1 Datasets

To evaluate Algorithms 5, 6, 7 and 8 on the ℓ^2-regularized logistic regression objective defined by Equations (1) and (2) we use the same three binary classification datasets as Mishchenko et al. (2020), namely real-sim ($n = 72{,}309$, $d = 20{,}958$), w8a ($n = 49{,}749$, $d = 300$) and RCV1 ($n = 804{,}414$, $d = 47{,}236$).[6] The RCV1 dataset is natively multi-label. We binarize it by taking the single most frequent class label as the positive class and mapping the remaining points to the negative class. In all three cases, the labels are mapped to $b_i \in \{-1, +1\}$ before training, as required by our objective (see Section 2.1).

For the experiments performed on neural networks (see Equations (3), (4), (5) and (6)) we use the Bike Sharing dataset (Fanaee-T and Gama 2013) for a first regression problem ($n = 17{,}379, d = 18$), the Concrete Compressive Strength dataset (Yeh 1998) for a second regression problem ($n = 1{,}030, d = 8$) and a random subsample of $5{,}000$ points of the MNIST dataset (LeCun et al. 1998) for the classification task ($n = 5{,}000$, $d = 784$).[7] For Bike Sharing, the four categorical features (`season`, `holiday`, `workingday`, `weather`) are

[5] This ensures that the edge case $\tau = 1$ exactly recovers the single-sample formulation of Algorithms 1, 2, 3 and 4.

[6] real-sim and w8a are loaded via `load_svmlight_file` from the copies distributed in Mishchenko et al. (2020)'s code release (https://github.com/konstmish/random_reshuffling). RCV1 is loaded via `sklearn.datasets.fetch_rcv1`.

[7] Loaded via `sklearn.datasets.fetch_openml` (Bike Sharing: `Bike_Sharing_Demand`, version 2; Concrete Compressive Strength: `Concrete_Data`, version 1) and `torchvision.datasets.MNIST` (MNIST). Both OpenML entries are re-hostings of the original UCI datasets.

Algorithm 5: Mini-Batch Stochastic Gradient Descent (SGD)

Input: Stepsizes (γ_t^j), batch size τ, normalization $\nu = \tau$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T

for *epochs* $t = 0, ..., T-1$ **do**
  for $j = 0, ..., m-1$ **do**
    Sample mini-batch S_j of size τ from $\{1, ..., n\}$ *with* replacement
    $\boldsymbol{x}_t^{j+1} = \boldsymbol{x}_t^j - \gamma_t^j \, g_\nu^j(\boldsymbol{x}_t^j)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^m$
end

Algorithm 6: Mini-Batch Random Reshuffling (RR)

Input: Stepsizes (γ_t^j), batch size τ, normalization $\nu = n/m$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T

for *epochs* $t = 0, ..., T-1$ **do**
  Sample a permutation $\pi_0, ..., \pi_{n-1}$ of $\{1, ..., n\}$
  Partition $\pi_0, ..., \pi_{n-1}$ into contiguous mini-batches $S_0, ..., S_{m-1}$ of sizes $\tau, ..., \tau, n-(m-1)\tau$
  for $j = 0, ..., m-1$ **do**
    $\boldsymbol{x}_t^{j+1} = \boldsymbol{x}_t^j - \gamma_t^j \, g_\nu^j(\boldsymbol{x}_t^j)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^m$
end

Algorithm 7: Mini-Batch Shuffle-Once (SO)

Input: Stepsizes (γ_t^j), batch size τ, normalization $\nu = n/m$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T

Sample a permutation $\pi_0, ..., \pi_{n-1}$ of $\{1, ..., n\}$
Partition $\pi_0, ..., \pi_{n-1}$ into contiguous mini-batches $S_0, ..., S_{m-1}$ of sizes $\tau, ..., \tau, n-(m-1)\tau$

for *epochs* $t = 0, ..., T-1$ **do**
  for $j = 0, ..., m-1$ **do**
    $\boldsymbol{x}_t^{j+1} = \boldsymbol{x}_t^j - \gamma_t^j \, g_\nu^j(\boldsymbol{x}_t^j)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^m$
end

Algorithm 8: Mini-Batch Incremental Gradient (IG)

Input: Stepsizes (γ_t^j), batch size τ, normalization $\nu = n/m$, initial vector $\boldsymbol{x}_0^0 \in \mathbb{R}^p$, number of epochs T

Partition $\{1, ..., n\}$ into contiguous mini-batches $S_0, ..., S_{m-1}$ of sizes $\tau, ..., \tau, n-(m-1)\tau$

for *epochs* $t = 0, ..., T-1$ **do**
  for $j = 0, ..., m-1$ **do**
    $\boldsymbol{x}_t^{j+1} = \boldsymbol{x}_t^j - \gamma_t^j \, g_\nu^j(\boldsymbol{x}_t^j)$
  end
  $\boldsymbol{x}_{t+1}^0 = \boldsymbol{x}_t^m$
end

one-hot encoded,[8] while the remaining numeric features are standardized and clipped to $[-5, 5]$. The regression target is likewise standardized before training. Concrete Compressive Strength has eight numeric features, with no categorical columns. We standardize and clip them to $[-5, 5]$ exactly as for Bike Sharing's numeric features and likewise standardize the regression target. We normalize the pixels of the MNIST dataset to $\boldsymbol{a}_i \in [0, 1]^d$ before training.

3.2 Reporting Convergence Curves

Every convergence curve shown in Section 4 aggregates across all seeds run for that experiment (20 for the ℓ^2-regularized logistic regression datasets, 5 for the neural network datasets). We plot the mean curve together with a shaded band, both computed in log-space. For each stored iterate, we take the mean μ_k and standard deviation σ_k of $\log(\text{value})$ across seeds, then plot $\exp(\mu_k)$ as the central curve and $\exp(\mu_k \pm \sigma_k)$ as the band edges. The central curve is thus the *geometric* mean across seeds and the band a ± 1 (multiplicative) standard deviation around it, which is also what keeps the band visually symmetric on our logarithmic y-axis. Note that this band reflects the spread of the individual seed trajectories, not a confidence interval for the mean. It does not shrink as more seeds are added and should not be read as a statement of statistical significance.

[8] `holiday` and `workingday` take only two values, so a standard one-hot encoding would produce two perfectly redundant columns (e.g. `is_holiday` and `is_not_holiday`, related by $\text{is_not_holiday} = 1 - \text{is_holiday}$). We keep only one indicator column per binary feature instead.

3.3 Hyperparameter Choices for the ℓ^2-Regularized Logistic Regression Experiments

To recreate the experiments of Mishchenko et al. (2020) we adopt their hyperparameter protocol. A full description and derivations for these choices are given in Sections 4 and 5 of their paper and the accompanying code release: https://github.com/konstmish/random_reshuffling. Hence, we only summarize the resulting formulas here:

Regularization strength is set to $\lambda = L/\sqrt{n}$, where $L = \|\boldsymbol{A}\|_2^2/(4n)$ is the spectral smoothness constant of the unregularized logistic loss and $\boldsymbol{A} = [\boldsymbol{a}_1, ..., \boldsymbol{a}_n]^\top$ is the data matrix. All methods use a fixed batch size $\tau = 512$ and a constant-then-decaying stepsize schedule defined as follows: for the first $k_0 = \lfloor K/40 \rfloor$ iterations $\gamma_k = \gamma_{\max} = 1/L_\tau$, where $K = mT$ is the total number of iterations and $L_\tau = \frac{n(\tau-1)}{\tau(n-1)} L_f + \frac{n-\tau}{\tau(n-1)} L_{\max}$ is the mini-batch smoothness constant given in Proposition 3.8 (iv) of Gower et al. (2019), with $L_f = L + \lambda$ and $L_{\max} = \max_i \|\boldsymbol{a}_i\|_2^2/4 + \lambda$ the largest per-sample smoothness constant of the ℓ^2-regularized logistic loss; afterwards $\gamma_k = \min\{\gamma_{\max}, 1/(c(k-k_0))\}$ with $c = \lambda/3$ for RR, SO and IG and $c = \lambda/2$ for SGD. Each dataset is trained for a different number of epochs, matching their reported horizon (150 for real-sim, 250 for w8a, 30 for RCV1), with 20 seeds per configuration. All methods are initialized at $\boldsymbol{x}_0^0 = \boldsymbol{0}$. Since the objective is strongly convex, we approximate the optimum $(\boldsymbol{x}^*, f^*)$ via 5,000 steps of (full-batch) gradient descent from $\boldsymbol{x}_0^0$. Here we deviate from Mishchenko et al. (2020) slightly, as they ran Nesterov's accelerated gradient method to machine precision. Since the objective is λ-strongly convex and L_f-smooth, gradient descent itself converges linearly to $\boldsymbol{x}^*$. In practice, it plateaued well before exhausting the step budget, confirming that 5,000 steps are more than enough to approximate the optimum.

3.4 Hyperparameter Choices for the Neural Network Experiments

On the three neural network datasets we train both network architectures (see Section 2.1) using a *constant* stepsize. One network model has two hidden layers of width 128 and 64. The other has no hidden layers at all (an affine map), isolating the effect of the ReLU non-linearities. On Bike Sharing (regression, ℓ^2-regularized MSE loss) we use batch size $\tau = 16$, regularization strength $\lambda = 10^{-4}$ and train for a horizon of 500 epochs. The two-hidden-layer network uses $\gamma = 5 \times 10^{-3}$, while the affine baseline runs on a smaller stepsize $\gamma = 10^{-4}$. On Concrete Compressive Strength (regression, ℓ^2-regularized MSE loss) we use batch size $\tau = 16$, regularization strength $\lambda = 10^{-4}$ and train for a horizon of 1,000 epochs. The two-hidden-layer network uses $\gamma = 5 \times 10^{-3}$, while the affine baseline again adopts a smaller stepsize $\gamma = 10^{-3}$. On MNIST (classification, ℓ^2-regularized softmax cross-entropy loss) we use batch size $\tau = 8$, regularization strength $\lambda = 10^{-3}$ and stepsize $\gamma = 5 \times 10^{-3}$ for both the two-hidden-layer network and the affine baseline and train for a horizon of 1,000 epochs. All settings use 5 seeds.

Unlike the ℓ^2-regularized logistic regression experiments, each seed initializes the network parameters from scratch using PyTorch's default initialization for the weights and biases of linear layers, where both are sampled from $\mathcal{U}(-\alpha, \alpha)$ with $\alpha = 1/\sqrt{\text{in_features}}$ (PyTorch Contributors 2026a), rather than initializing the parameter vector as $\boldsymbol{x}_0^0 = \boldsymbol{0}$. Since the resulting objective is generally non-convex, no closed-form or provably global optimum exists. Instead, we take the best final iterate reached by any of the four methods across all seeds and polish it further with 2,000 additional full-batch gradient steps at the same stepsize γ to find $\tilde{f}^*$.[9] The reported gap $f(\boldsymbol{x}_k) - \tilde{f}^*$ is therefore a gap to the best loss *found*.

4 Results and Discussion

Turning now to the empirical results, we structure the discussion around the two settings: strongly convex ℓ^2-regularized logistic regression, where we recreate the findings of Mishchenko et al. (2020), and neural network training, where we compare strongly convex (affine) and non-convex (two-hidden-layer) architectures side by side and find that the same four algorithms behave quite differently.

4.1 ℓ^2-Regularized Logistic Regression: Recreating the Behavior in Mishchenko et al. (2020)

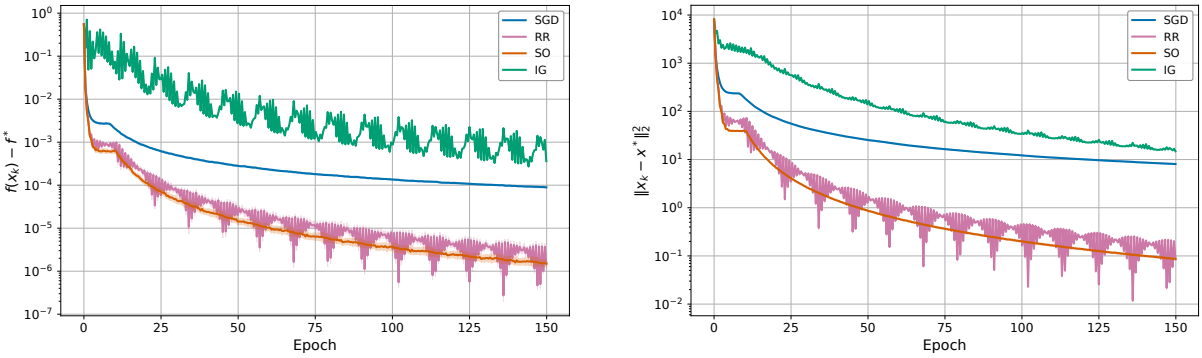
Figures 1, 2 and 3 show the convergence trajectories of the four algorithms SGD, RR, SO and IG used to solve the ℓ^2-regularized logistic regression problem on the datasets real-sim, w8a and RCV1, respectively. While

[9] The one exception is the affine baseline on Concrete Compressive Strength, where we polish with $\gamma = 2 \times 10^{-1}$ instead. At the training stepsize $\gamma = 10^{-3}$, 2,000 GD steps barely contracted the remaining gap.

the (a) panels show the optimality gap $f(\boldsymbol{x}_k) - f^*$, the (b) panels display the squared Euclidean norm of the iterate gap $\|\boldsymbol{x}_k - \boldsymbol{x}^*\|_2^2$.

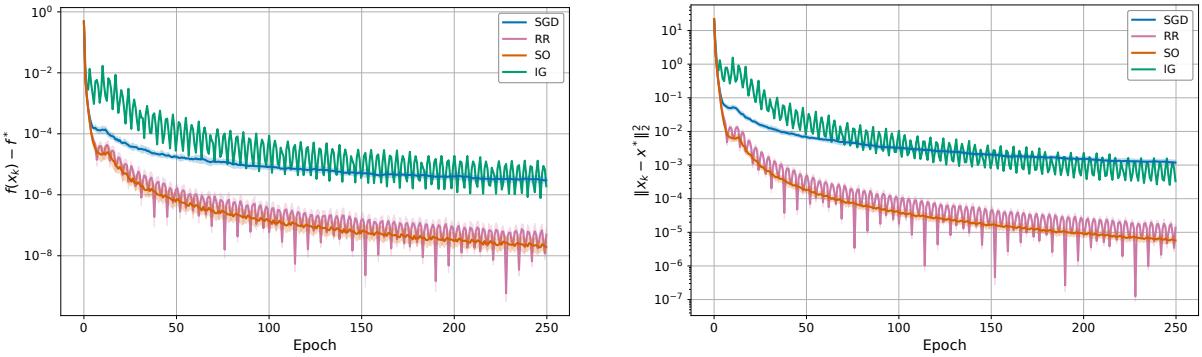
All six plots tell the same story and it is the story from Figure 1 of Mishchenko et al. (2020): RR and SO track each other closely and beat SGD and IG by a wide margin. This close match between RR and SO is unsurprising as Mishchenko et al. (2020)'s Theorem 1 covers both algorithms with the same bound. Hence, their similar convergence speed is exactly what the theory predicts. When it comes to IG's performance relative to SGD, we observe that this depends on the dataset, the number of epochs and the metric used for the comparison. IG sometimes performs worse than SGD, for instance on the optimality gap for real-sim (see Figure 1a), but can also converge faster than SGD, as seen in the iterate gap for RCV1 (see Figure 3b).

It is worth emphasizing that RR and SO consistently outperform IG across all of our ℓ^2-regularized logistic regression experiments. This provides further empirical support for what Bertsekas (2015) had already noticed: there exist permutations of the dataset along which gradient evaluations are unfavorable. Moreover, it also reflects a broader trend in modern numerical computing. Probabilistic and randomized approaches to algorithm design, from randomized numerical linear algebra (Martinsson and Tropp 2020) to stochastic optimization methods like the ones studied here, have been shown to outperform their deterministic counterparts, whether in terms of computational efficiency, scalability or empirical solution quality. (We recall here the case made in Section 1 against vanilla gradient descent and in favor of stochastic variants.)



(a) Optimality gap (b) Iterate gap

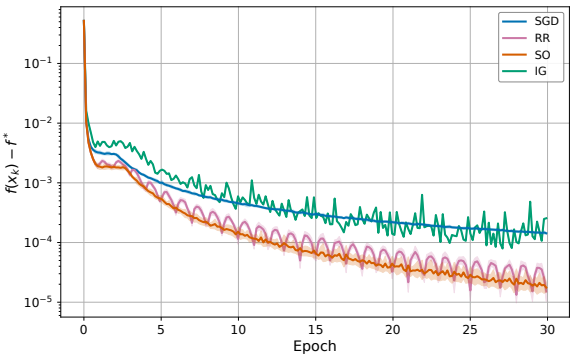
Figure 1: Convergence on real-sim ($n = 72{,}309, d = 20{,}958$) for ℓ^2-regularized logistic regression. Hyperparameters: $\tau = 512, \gamma_{\max} = 272, \lambda = 1.18 \times 10^{-5}$.



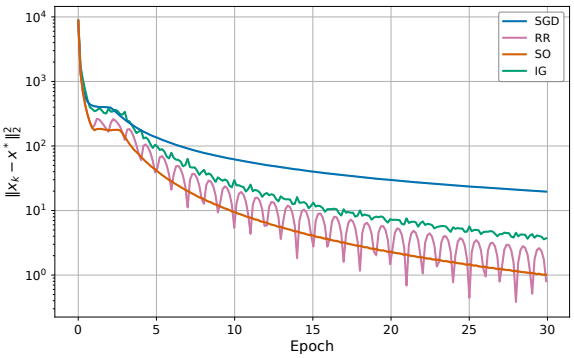
(a) Optimality gap (b) Iterate gap

Figure 2: Convergence on w8a ($n = 49{,}749, d = 300$) for ℓ^2-regularized logistic regression. Hyperparameters: $\tau = 512, \gamma_{\max} = 1.39, \lambda = 2.96 \times 10^{-3}$.

Lastly, we would like to draw attention to the oscillations visible in the trajectories of all *without*-replacement schemes. Although the proof of Theorem 1 in Mishchenko et al. (2020) predicts that the iterates drift away from $\boldsymbol{x}^*$ before snapping back near the epoch boundary, it is nevertheless fascinating to observe how large these oscillations become, particularly for RR, and the characteristic sawtooth patterns that emerge. The apparent period of these oscillations differs somewhat from what is visible in Figure 1 of Mishchenko et al. (2020). For instance, our periods for RR and IG on real-sim (see Figure 1) are slightly shorter than theirs. As



(a) Optimality gap



(b) Iterate gap

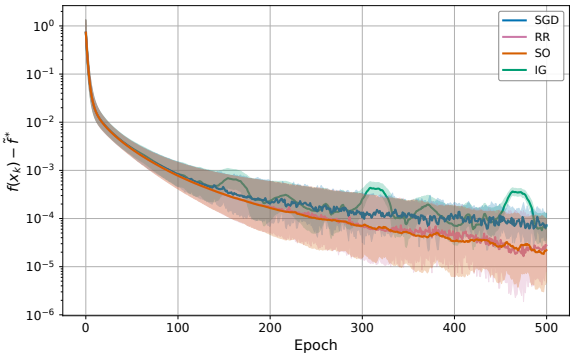
Figure 3: Convergence on RCV1 ($n = 804{,}414, d = 47{,}236$) for ℓ^2-regularized logistic regression. Hyperparameters: $\tau = 512, \gamma_{\max} = 154, \lambda = 6.72 \times 10^{-6}$.

mentioned before, the true oscillation has a period of exactly one epoch (Mishchenko et al. 2020, Theorem 1), but our figures only show a handful of stored points per epoch, controlled by the `plot_every` parameter. Consequently, how many points are plotted within a single epoch determines how clean the sawtooth pattern looks. As a direct check, we re-plotted the curves for w8a at a higher point density[10] and found its oscillation pattern closely resembles real-sim's. We therefore attribute the differing apparent period to plotting density, not to any underlying difference in the implementation of the algorithms themselves.

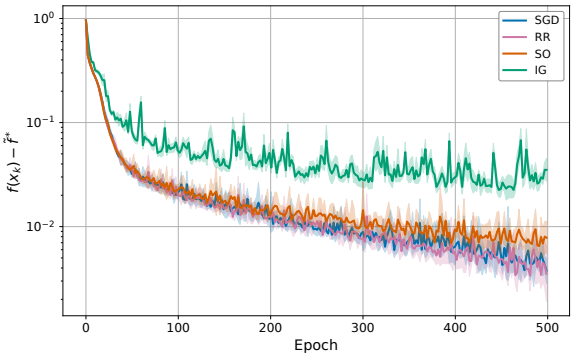
4.2 Neural Networks: Moving from Strong Convexity to Non-Convexity

In Figures 4, 5 and 6 we show the convergence results of the four algorithms SGD, RR, SO and IG used to solve the neural network regression problems on the Bike Sharing and Concrete Compressive Strength datasets and the neural network classification problem on MNIST. On the left, the (a) panels present the strongly convex problem settings where no hidden layers are included in the network architecture, leading to an affine model. On the right, the (b) panels correspond to the two-hidden-layer network architecture whose ReLU activations make the minimization objective non-convex.

Like in the ℓ^2-regularized logistic regression plots, we notice that the convergence trajectories of RR and SO on the zero-hidden-layer networks mostly stick together (see Figures 4a, 5a and 6a). Also, RR and SO outperform SGD throughout in the strongly convex setting. When it comes to IG, we again see that it depends on the dataset and the number of epochs how the algorithm compares to SGD. Notably, IG mimics the convergence speed of RR and SO in Figure 6a. As it turns out, this is because our fixed-seed subsampling already applies a random permutation to the MNIST data before training, so IG's natural-index traversal is effectively a random permutation too, just like SO's.



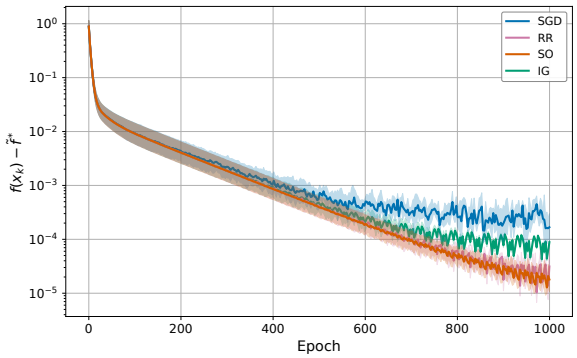
(a) 0-hidden-layer (affine) NN architecture ($18 \to 1$). Hyperparameters: $\tau = 16, \gamma = 10^{-4}, \lambda = 10^{-4}$.



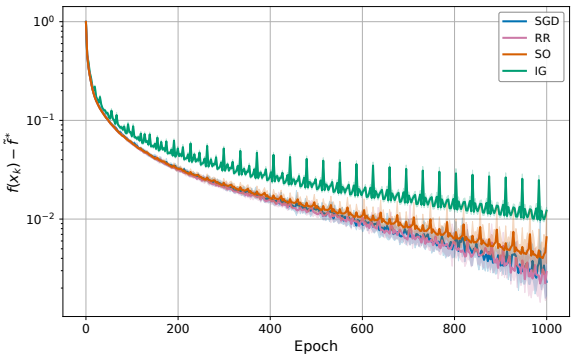
(b) 2-hidden-layer NN architecture ($18 \to 128 \to 64 \to 1$). Hyperparameters: $\tau = 16, \gamma = 5 \times 10^{-3}, \lambda = 10^{-4}$.

Figure 4: Optimality gap to best training loss found on Bike Sharing ($n = 17{,}379, d = 18$).

[10] For reproducibility: `plot_every = 12 * store_every`

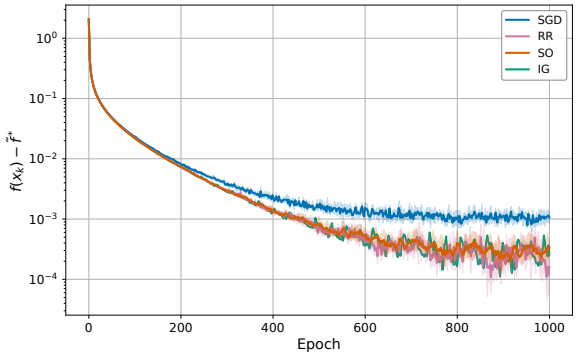


(a) 0-hidden-layer (affine) NN architecture ($8 \to 1$). Hyperparameters: $\tau = 16, \gamma = 10^{-3}, \lambda = 10^{-4}$.

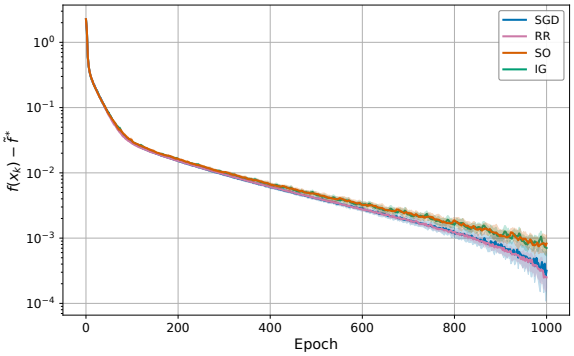


(b) 2-hidden-layer NN architecture ($8 \to 128 \to 64 \to 1$). Hyperparameters: $\tau = 16, \gamma = 5 \times 10^{-3}, \lambda = 10^{-4}$.

Figure 5: Optimality gap to best training loss found on Concrete Compressive Strength ($n = 1{,}030, d = 8$).



(a) 0-hidden-layer (affine) NN architecture ($784 \to 10$). Hyperparameters: $\tau = 8, \gamma = 5 \times 10^{-3}, \lambda = 10^{-3}$.



(b) 2-hidden-layer NN architecture ($784 \to 128 \to 64 \to 10$). Hyperparameters: $\tau = 8, \gamma = 5 \times 10^{-3}, \lambda = 10^{-3}$.

Figure 6: Optimality gap to best training loss found on subsampled MNIST ($n = 5{,}000, d = 784$).

Things start to turn more interesting, however, when we leave the strongly convex setting and focus on the (b) panels. The drawback from which SGD suffered in the former case completely disappears in the non-convex neural network setting. Across all three datasets, there is a performance difference between SGD and RR in the affine (strongly convex) baseline, but not so much in the two-hidden-layer (non-convex) network. This suggests that RR's edge over SGD is tied to strong convexity and vanishes once that assumption is dropped. It is impressive to see that across Figures 4b, 5b and 6b it is now SGD (and no longer SO) which tracks RR. Indeed, SO starts to lag behind; so much so that in the case of MNIST it even degenerates into the convergence performance of IG, which exhibits the worst performance out of all algorithms in all two-hidden-layer NN experiments. We believe this role reversal is best explained by returning to the probabilistic argument for algorithm design raised in Section 4.1: what separates SGD and RR from SO and IG here is the amount of *fresh* stochasticity injected into the sampling order at every epoch. On the one hand, RR reshuffles the dataset anew each epoch and SGD resamples with replacement at every single iteration. On the other hand, SO and IG each commit to one fixed traversal of the dataset and repeat it identically for all of training. This means that while SO and IG retrace the same trajectory through the loss landscape data pass after data pass, RR and SGD are exposed to a different one each epoch. We conjecture that it is this repeated re-randomization that lets SGD and RR navigate a complex non-convex loss landscape more effectively and avoid getting stuck in the same local structures that SO and IG cannot help but revisit.

4.3 Further Observations and Caveats on the Neural Network Experiments

We note that initializing x_0^0 according to PyTorch's default for linear layers (see Section 3.4) is what produces the visible shaded band around IG's curve in the neural network plots. The ordering IG iterates along is a fixed cyclic pass through the data with no random component, so its five trajectories differ only because each seed starts from a different initialization. This divergence is exactly what the band in Section 3.2 then captures.

More broadly, the width of the bands themselves calls for caution rather than over-interpretation: with

only 5 seeds (against 20 for the ℓ^2-regularized logistic regression datasets) and fresh per-seed initialization (rather than a shared $\boldsymbol{x}_0^0 = \mathbf{0}$), the NN rankings should be read as suggestive, not as tightly resolved as the ones in Section 4.1.

While the ℓ^2-regularized logistic regression plots in Figures 1, 2 and 3 show clearly visible oscillations, the same behavior in the neural network plots is far more subtle and in some cases entirely absent. In the affine MNIST case (see Figure 6a), both SO and IG do in fact show oscillatory behavior on closer look, though for RR this remains unclear from the plot. In the two-hidden-layer MNIST case (see Figure 6b), the amplitude of any such oscillation is too small relative to the y-axis scaling to tell one way or the other. For Concrete Compressive Strength (see Figures 5a and 5b), oscillations are clearly visible for both SO and IG, while again RR shows no apparent pattern. For Bike Sharing (see Figures 4a and 4b), we cannot identify a genuine oscillatory pattern for any of the four algorithms, with the exception of IG in the affine case, which exhibits a single pronouncedly long and high-amplitude oscillation.

This inconsistency is puzzling in particular for the affine architecture. As its objective is strongly convex, Theorem 1 of Mishchenko et al. (2020) predicts the same kind of oscillation should be present there too, not just occasionally visible for SO and IG. It is not clear to us why oscillations are visible in some of these plots but not others. One conjecture is that this is at least partly an artifact of scale rather than a difference in underlying behavior. All three neural network experiments are trained for substantially more epochs (500–1,000) than the ℓ^2-regularized logistic regression experiments (30–250). Squeezing this many epochs into the same fixed horizontal plot width, combined with the correspondingly lower plotting density this requires, may compress and thereby hide oscillations that are in fact present throughout. However, the networks on Concrete Compressive Strength and MNIST are both trained for the same 1,000 epochs yet show very different oscillation visibility, so whatever role epoch count plays likely interacts with dataset-specific properties as well. A second, related possibility is the aforementioned per-seed initialization. Since each seed now starts from a different point rather than a shared $\boldsymbol{x}_0^0 = \mathbf{0}$, any oscillation specific to one seed's trajectory could be smoothed out once the mean curve is aggregated across seeds, even if that oscillation were present in any individual run.

The gap to best-found loss plots alone make the affine baseline look like the better-converging model, but Table 1 tells the actual story: the two-hidden-layer network reaches a floor roughly 20 to 17 times lower than the affine baseline on Bike Sharing and Concrete Compressive Strength, respectively, and over 3 times lower on MNIST. Hence, the two-hidden-layer neural network benefits from its increased complexity, indicating that the additional parameters are effectively being utilized. Since each curve is measured against its *own* $\tilde{f}^*$, the affine baseline's smaller gap only reflects that it is chasing an easier-to-reach best-found floor $\tilde{f}^*$. Rankings within a plot are meaningful, gap sizes across architectures are not.

Experiment	$\tilde{f}^*$
Bike Sharing (two-hidden-layer)	0.030506
Bike Sharing (zero-hidden-layer (affine))	0.601305
Concrete Compressive Strength (two-hidden-layer)	0.022570
Concrete Compressive Strength (zero-hidden-layer (affine))	0.383803
MNIST (two-hidden-layer)	0.075045
MNIST (zero-hidden-layer (affine))	0.269892

Table 1: Raw training loss values of the best-found loss $\tilde{f}^*$ reached by each NN architecture.

5 Conclusions

In this work we compared the four stochastic first-order methods SGD, RR, SO and IG on finite-sum minimization problems in two regimes. We first recreated some of the strongly convex ℓ^2-regularized logistic regression experiments of Mishchenko et al. (2020) on real-sim, w8a and RCV1. Then we extended the comparison to neural networks, covering regression on Bike Sharing and Concrete Compressive Strength and classification on a subsample of MNIST, each trained both as an affine baseline and as a two-hidden-layer ReLU network.

In the strongly convex regime our results confirm the picture painted by Mishchenko et al. (2020). RR and SO track each other closely and beat SGD by a wide margin, exactly as their Theorem 1 predicts. IG is less reliable. Depending on the dataset and metric it lands ahead of or behind SGD, but it never catches RR or SO. We also reproduce the epoch-wise oscillations of the without-replacement schemes, whose differences apparent period we attribute to plotting density rather than algorithm behavior.

The non-convex experiments tell a different story. Here SGD stages a comeback and catches up to RR, while SO falls behind and IG delivers the worst performance across all two-hidden-layer experiments. RR's edge over SGD thus appears tied to strong convexity and vanishes once that assumption is dropped. We attribute the role reversal between SGD and SO to the amount of fresh stochasticity each method injects: RR and SGD re-randomize their sampling order throughout training, whereas SO and IG retrace one fixed trajectory through the loss landscape pass after pass.

Two broader lessons run through all of our experiments. Order matters, as the same gradients applied in a different sequence produce measurably different convergence behavior. And stochasticity matters, since the methods that keep injecting randomness are the ones that cope best when the landscape becomes non-convex. Taken together, our results single out one method: RR was tied for best in every experiment we ran and never lost to another algorithm, neither under strong convexity nor in the non-convex setting. Given that reshuffling is a one-flag default in modern frameworks, we see little reason not to make RR the method of choice for finite-sum minimization.

6 Future Work

Three natural extensions could deepen or broaden the comparison made in this work. The first direction is to investigate more systematically why the oscillatory behavior visible in the ℓ^2-regularized logistic regression plots appears inconsistently across the neural network experiments, present and clearly visible for some datasets and architectures, subtle or absent for others. A study isolating the roles of plotting density, per-seed initialization and dataset-specific properties would help settle whether this is a genuine difference in algorithm behavior or a byproduct of how the curves are aggregated and plotted.

A second direction stays close to RR itself. Shaw and Whalley (2025) recently studied an alternating-direction variant through the lens of splitting methods: after sampling a new permutation, the method takes a full forward pass through it, followed by a full backward pass through the same permutation, before reshuffling and repeating. They show it can offer improved higher-order convergence guarantees under the right normalization and a decaying stepsize schedule. A dedicated empirical comparison in the neural network setting would be a natural next step.

A third direction is to test how far our neural network results generalize beyond the deliberately simple setup used here. Larger, deeper and different architectures and the removal of the ℓ^2-regularization term we impose throughout would all help clarify whether our observations are specific to our setup or if they hold more broadly.

Acknowledgements

I would like to thank Dr. Peter Whalley for supervising this project, suggesting its topic and his valuable feedback and patience throughout. I am also grateful to Prof. Dr. Yuansi Chen for giving me the opportunity to work on this paper in his group.

References

Yoshua Bengio (2012). "Practical Recommendations for Gradient-Based Training of Deep Architectures". In: *Neural Networks: Tricks of the Trade: Second Edition*. Ed. by Grégoire Montavon, Geneviève B. Orr, and Klaus-Robert Müller. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 437–478. ISBN: 978-3-642-35289-8. DOI: 10.1007/978-3-642-35289-8_26. URL: https://doi.org/10.1007/978-3-642-35289-8_26.

Dimitri Bertsekas (July 2015). "Incremental Gradient, Subgradient, and Proximal Methods for Convex Optimization: A Survey". In: *Optimization* 2010.

Dimitri Bertsekas and John N. Tsitsiklis (2000). "Gradient Convergence in Gradient methods with Errors". In: *SIAM Journal on Optimization* 10.3, pp. 627–642. DOI: 10.1137/S1052623497331063. eprint: https://doi.org/10.1137/S1052623497331063. URL: https://doi.org/10.1137/S1052623497331063.

Léon Bottou (2009). "Curiously fast convergence of some stochastic gradient descent algorithms". Unpublished open problem offered to the attendance of the SLDS 2009 conference. URL: http://leon.bottou.org/papers/bottou-slds-open-problem-2009.

Léon Bottou, Frank E. Curtis, and Jorge Nocedal (2018). "Optimization Methods for Large-Scale Machine Learning". In: *SIAM Review* 60.2, pp. 223–311. DOI: 10.1137/16M1080173. eprint: https://doi.org/10.1137/16M1080173. URL: https://doi.org/10.1137/16M1080173.

Hadi Fanaee-T and João Gama (2013). "Event labeling combining ensemble detectors and background knowledge". In: *Progress in Artificial Intelligence*, pp. 1–15. DOI: 10.1007/s13748-013-0040-3.

Robert Mansel Gower et al. (Sept. 2019). "SGD: General Analysis and Improved Rates". In: *Proceedings of the 36th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri and Ruslan Salakhutdinov. Vol. 97. Proceedings of Machine Learning Research. PMLR, pp. 5200–5209. URL: https://proceedings.mlr.press/v97/qian19b.html.

Mert Gürbüzbalaban, Asuman Ozdaglar, and Pablo A. Parrilo (2021). "Why random reshuffling beats stochastic gradient descent". In: *Mathematical Programming* 186, pp. 49–84.

Jeff Z. HaoChen and Suvrit Sra (2018). "Random Shuffling Beats SGD after Finite Epochs". In: *arXiv preprint arXiv:1806.10077*.

Yann LeCun et al. (1998). "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324. DOI: 10.1109/5.726791.

Zijian Liu (2026). *Random Reshuffling Dominates Stochastic Gradient Descent*. COLT 2026. arXiv: 2606.32005 [math.OC]. URL: https://arxiv.org/abs/2606.32005.

Grigory Malinovsky, Alibek Sailanbayev, and Peter Richtárik (2023). "Random Reshuffling with Variance Reduction: New Analysis and Better Rates". In: *Proceedings of the Thirty-Ninth Conference on Uncertainty in Artificial Intelligence*. Vol. 216. Proceedings of Machine Learning Research. PMLR, pp. 1347–1357. URL: https://proceedings.mlr.press/v216/malinovsky23a.html.

Per-Gunnar Martinsson and Joel A. Tropp (2020). "Randomized numerical linear algebra: Foundations and algorithms". In: *Acta Numerica* 29, pp. 403–572. URL: https://api.semanticscholar.org/CorpusID:229167320.

Konstantin Mishchenko, Ahmed Khaled, and Peter Richtárik (2020). "Random Reshuffling: Simple Analysis with Vast Improvements". In: *Advances in Neural Information Processing Systems* 33. arXiv:2006.05988.

PyTorch Contributors (2026a). *torch.nn.Linear*. URL: https://docs.pytorch.org/docs/2.13/generated/torch.nn.Linear.html#torch.nn.Linear (visited on 07/14/2026).

— (2026b). *torch.utils.data*. URL: https://docs.pytorch.org/docs/2.13/data.html (visited on 07/16/2026).

Herbert Robbins and Sutton Monro (1951). "A Stochastic Approximation Method". In: *The Annals of Mathematical Statistics* 22.3, pp. 400–407. ISSN: 00034851. URL: http://www.jstor.org/stable/2236626 (visited on 07/16/2026).

Luke Shaw and Peter A. Whalley (2025). *Randomised Splitting Methods and Stochastic Gradient Descent*. arXiv: 2504.04274 [math.OC]. URL: https://arxiv.org/abs/2504.04274.

Ruoyu Sun (June 2020). "Optimization for Deep Learning: An Overview". English (US). In: *Journal of the Operations Research Society of China* 8.2. Publisher Copyright: © 2020, Operations Research Society of China, Periodicals Agency of Shanghai University, Science Press, and Springer-Verlag GmbH Germany, part of Springer Nature., pp. 249–294. ISSN: 2194-668X. DOI: 10.1007/s40305-020-00309-6.

I-Cheng Yeh (1998). "Modeling of strength of high-performance concrete using artificial neural networks". In: *Cement and Concrete Research* 28.12, pp. 1797–1808. DOI: 10.1016/S0008-8846(98)00165-3.

Bicheng Ying et al. (Jan. 2019). "Stochastic Learning Under Random Reshuffling With Constant Step-Sizes". In: *IEEE Transactions on Signal Processing* 67.2, pp. 474–489. ISSN: 1941-0476. DOI: 10.1109/tsp.2018.2878551. URL: http://dx.doi.org/10.1109/TSP.2018.2878551.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Order Matters, Randomness Too: Data Sampling Strategies for Finite-Sum Minimization

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**
Frei	Basil

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**
Küsnacht ZH, July 20, 2026	B. Frei

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).